

Java – Beautifully Broken

The JVM learns to keep its homework, and Spring moves to one patch day a month

Issue #2

Tue, 06 Oct 2026

~33 min read

Level 3/5

Today's two stories

1. **JEP 544, "Ahead-of-Time Code Compilation", is now targeted to JDK 28.** The JVM will be able to save the optimised machine code from a training run and load it instantly the next time your app starts, so services start and reach full speed much sooner, with no code changes.
2. **Spring now ships all its patches on one day a month, "Patch Thursday", starting 22 October.** It's a response to an AI-driven flood of security reports (482 in April alone, against a historical average of about 6.5 a month), and it changes how every Spring team should plan upgrades.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 JEP 544: the JVM stops throwing away its best work

JDK 28 · OPENJDK · PROJECT LEYDEN · TARGETED TO JDK 28 (WEEK OF 28 SEP) · ~16 MIN READ

Every time a Java service starts, the JVM does something slightly absurd. It spends the first seconds or minutes watching your code run, works out which methods are hot, and compiles them into fast machine code. Then the pod is replaced, and all of that work is thrown away. The next instance starts from zero and does exactly the same thing again. JEP 544 ends that ritual. In JDK 28 the JVM will be able to save its compiled code to disk and pick it up the moment it starts. It's the step that Project Leyden, OpenJDK's six-year effort to make Java start fast, has been building toward since 2020.

WHAT HAPPENED

During the week of 28 September, **JEP 544, "Ahead-of-Time Code Compilation", moved from Proposed to Target to Targeted for JDK 28**, InfoQ reported on 5 October.¹ Its stated goal is to "improve startup and warmup time by making optimized native code for an application instantly available when the HotSpot Java Virtual Machine starts".² JDK 28 is the next feature release, due in March 2027. It's a non-LTS release with six months of Oracle support.³

The JEP moved fast. It was promoted from a draft (JDK-8335368) to Candidate in early September,⁴ was listed as proposed for JDK 28 by 24 September,³ and was targeted a week later.¹ The implementation is in pull request #30778 on the openjdk/jdk repository, "Implement JEP 544: Ahead-of-Time Code Compilation", by Vladimir Kozlov. When we checked it, the pull request was still open, with review in progress and a companion

CSR ≈ Compatibility and Specification Review: the OpenJDK process that approves changes to public behaviour, such as new command-line options request (JDK-8380477) still awaiting approval.⁵

Tool vendors are already preparing. On 5 October, the Eclipse Java compiler team opened a tracking issue for JEP 544 under its Java 28 milestone.²

THE BACKGROUND YOU NEED

Why Java starts slow: interpreter first, compiler later

To see why this matters, you need to know what the JVM does with your code. `javac` doesn't produce machine code. It produces **bytecode** ≈ the portable, CPU-independent instructions in .class files that the JVM executes. When your app starts, the JVM runs that bytecode with an **interpreter** ≈ a component that executes bytecode one instruction at a time. Simple and immediate, but much slower than native machine code.

While it interprets, the JVM watches. It counts how often each method runs and records what actually happens inside it: which branches are taken, which concrete classes show up at each call site. That record is called a **profile** ≈ the statistics the JVM gathers about how a method behaves at run time, used to decide what and how to optimise. Once a method is clearly hot, the **JIT compiler** ≈ the just-in-time compiler: the part of the JVM that compiles hot bytecode into native machine code while the program runs compiles it into **native code** ≈ machine instructions for the specific CPU, which run directly on the hardware.

HotSpot, the JVM inside nearly every JDK you use, has two JIT compilers working in tiers. **C1** ≈ HotSpot's fast "client" JIT compiler, which produces decent code quickly produces reasonable code quickly. **C2** ≈ HotSpot's optimising "server" JIT compiler, which uses profiles to produce the fastest code, but needs time and CPU uses the profile to produce highly optimised code, for example by **inlining** ≈ copying a called method's body into the caller to avoid the cost of the call and unlock further optimisations the methods you actually call and skipping branches that never happen. Moving through those levels is called **tiered compilation** ≈ HotSpot's strategy of running code first in the interpreter, then with quick C1 code, then with optimised C2 code as it gets hotter.

The period before the important methods reach C2 is called **warmup** ≈ the time after startup during which an application runs slower than its peak because the JIT has not yet compiled its hot code. JEP 515's authors call this a "chicken-and-egg problem": the JVM can't optimise well until it has observed the application, and observing takes time.⁶ JEP 483 adds that server applications built on frameworks like Spring can take seconds, or even minutes, just to start.⁷

For a long-running monolith, nobody cared. For a fleet of containers that scale up under load, it matters a lot. A fresh pod is slowest at exactly the moment it's needed most, and it burns CPU on compilation while it should be serving traffic.

The first attempt: jaotc, and why it died

Java tried **ahead-of-time compilation** ≈ compiling code to native machine code before the program runs, instead of while it runs once before. JDK 9 shipped an experimental tool called `jaotc` (JEP 295), and JDK 10 added the Java-based Graal compiler as an experimental JIT. In JDK 17, JEP 410 removed both. Its explanation was blunt: "We have seen little use of these experimental features since they were introduced, and the effort required to

maintain and enhance them is significant." Oracle hadn't even included them in its JDK 16 builds, "and no one complained."⁸ The removal deleted 26,703 lines of code across 375 files.⁹ The engineer who posted that removal, Vladimir Kozlov, is the author of the JEP 544 implementation five years later.^{9,5}

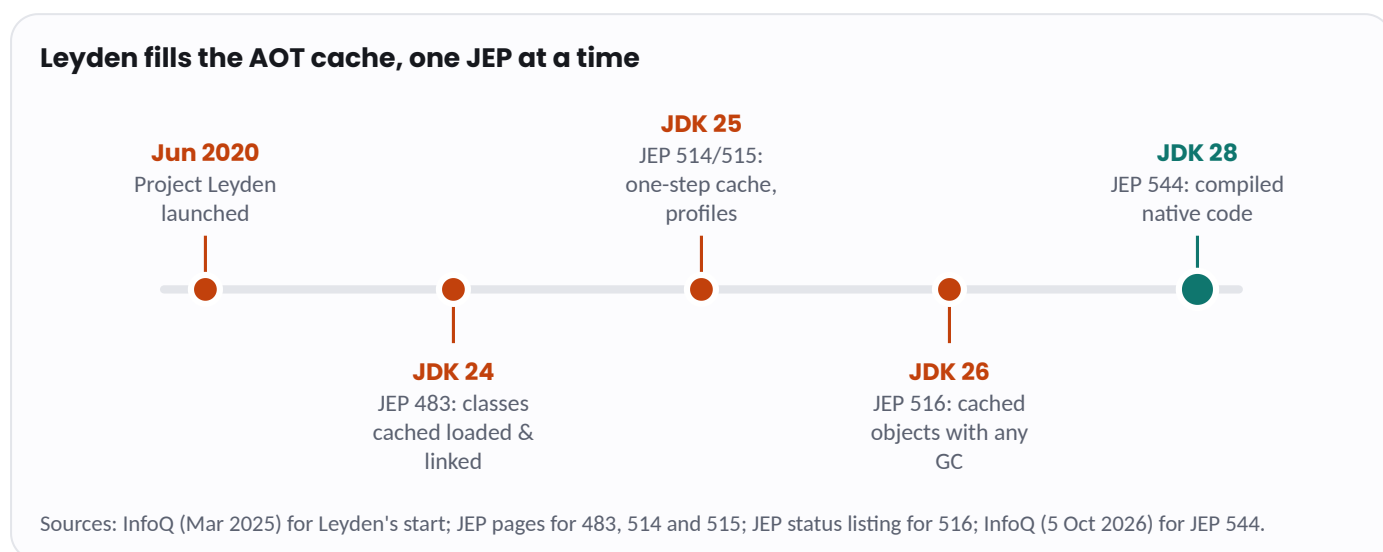
The other well-known route is **GraalVM Native Image** *≈ a tool that compiles a Java application into a standalone native executable ahead of time, under a "closed world" assumption that all code is known at build time*. It starts extremely fast, but it gives up some of Java's dynamism. Reflection, dynamic class loading and proxies need extra configuration, and you lose the profile-guided C2 compiler at run time.

Leyden: shift the work, don't change the language

Project Leyden launched in June 2020 to "improve the startup time, time to peak performance, and footprint of Java programs".^{10,11} At first it aimed at native-image-style static images. In June 2022 it changed direction: instead of a new closed-world model, it would gradually move work the JVM does at startup into an earlier run, keeping full Java semantics.¹⁰

The foundation already existed. **Class Data Sharing (CDS)** *≈ a JVM feature, available since JDK 5, that stores parsed class metadata in a file so later JVMs can map it in instead of re-reading and re-parsing the classes* has been around for years.⁷

Leyden turned it into a general-purpose **AOT cache** *≈ a file produced from a training run that stores work the JVM would otherwise redo at every startup: loaded classes, profiles and now compiled code* and added to it, one JEP per release:



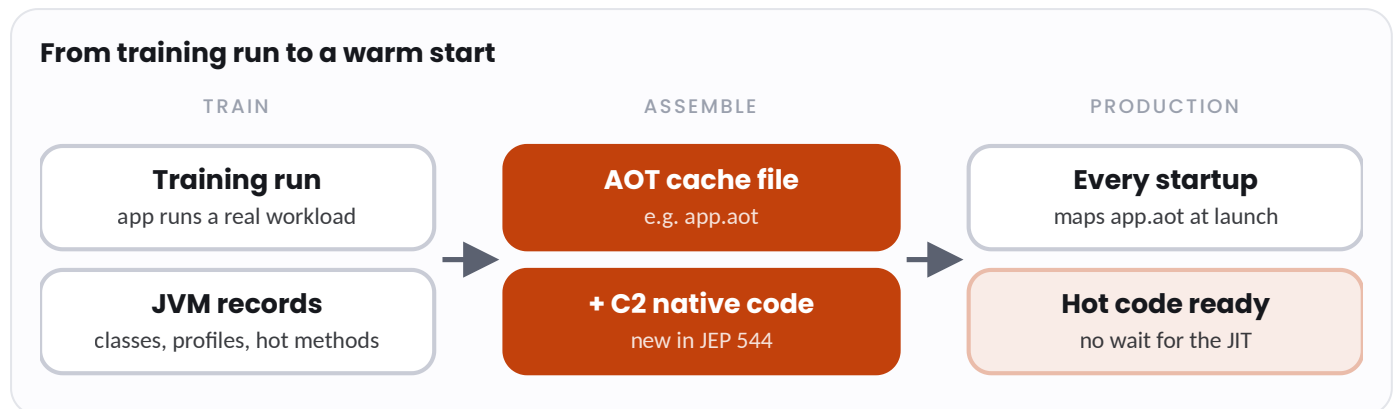
Each step attacked one piece of startup work. **JEP 483** (JDK 24) caches classes already loaded and **linked** *≈ the JVM step that verifies a class and resolves its references to other classes, before it can be used*. On Spring PetClinic, the JEP's own example, startup dropped from 4.486 s to 2.604 s, which is 42% faster.⁷ **JEP 514** (JDK 25) collapsed cache creation into one command.¹² **JEP 515** (JDK 25) added profiles to the cache, so the JIT can start compiling the right methods immediately. In its demo, that cut run time by 19%, from 90 ms to 73 ms.⁶ **JEP 516** (JDK 26) let cached heap objects work with any garbage collector.¹³

But until now, one expensive job was still done fresh in every production run: the compiling itself. JDK 25's cache tells C2 *what* to compile from the first second, yet C2 still has to spend CPU doing it. JEP 544 is the step that removes that.

HOW IT WORKS

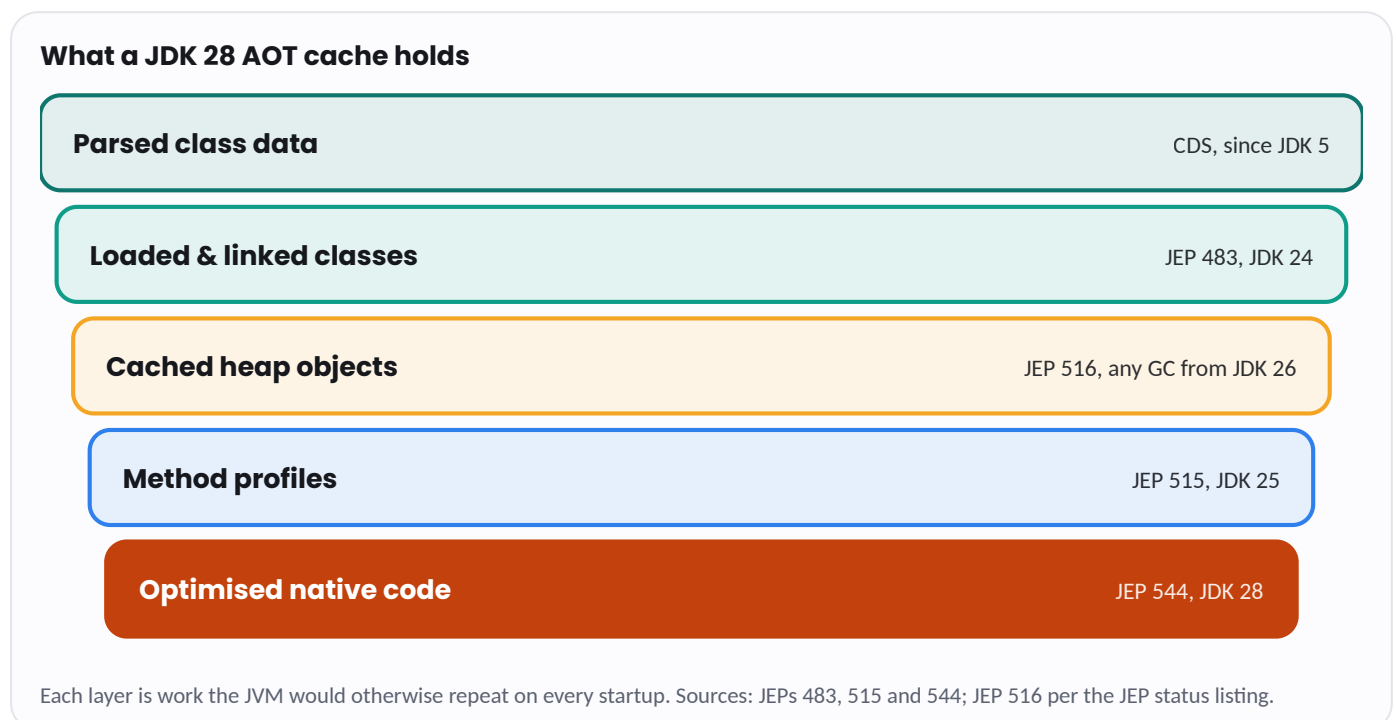
Three runs instead of one

The workflow is the one Leyden introduced in JDK 24, now with more inside the cache. You run your application once as a **training run** *≈ a run of your application, on a representative workload, whose only purpose is to record what the JVM learns so it can be saved into the AOT cache*. The JVM then assembles everything it learned into the cache file. Every later production run loads that file at startup.^{7,12}



What's inside the cache now

Think of the AOT cache as a growing stack of finished homework. Each Leyden JEP added a layer, and JEP 544 adds the most expensive one:



When a production JVM needs native code for a method, the request "normally fulfilled by the JIT compiler, can be immediately fulfilled if a matching method is found in the AOT cache", in the words of the JEP's draft.¹⁴ The code is produced by C2 during the training run, so it's optimised, profile-guided code.^{15,17}

The commands

Since JDK 25 the first two steps are a single command. With `-XX:AOTCacheOutput`, the launcher runs the training run and then creates the cache, using a temporary configuration file it deletes afterwards.¹² These are the commands from JEP 514. Secondary summaries of JEP 544 show it using the same ones; the cache simply holds more.^{12,17,18}

```
# JDK 25+: training run + cache creation in one step
java -XX:AOTCacheOutput=app.aot -cp app.jar com.example.App

# Every production start
java -XX:AOTCache=app.aot -cp app.jar com.example.App
```

The best of both worlds: it can still change its mind

This is what separates JEP 544 from a classic static compiler. The JEP's summary promises that Java will "gain some of the benefits of static compilation while retaining the agility, portability, and compatibility of dynamic compilation."¹⁶ The cached code is a head start, not a cage. If production behaves differently from training, the JVM can still **deoptimize** *≈ throw away compiled code whose assumptions turned out to be wrong and fall back to the interpreter, so it can re-profile and recompile* and recompile with the JIT, just as it always has.^{2,15} Interpreted bytecode, JIT code and AOT code all run side by side.¹⁴

Where it works, and where it quietly doesn't

Native code is tied to hardware in a way bytecode isn't, so AOT code comes with conditions. According to the JEP, as summarised by daily.dev:¹⁵

- **CPUs:** AArch64 and x64 only, at first.
- **Garbage collectors:** Serial, Parallel, G1 and ZGC.
- **Same hardware and GC:** the training and production runs must match in CPU architecture, CPU feature set and garbage collector. Otherwise the JVM ignores the cached code and goes back to interpreting and JIT-compiling.

The GC rule exists because compiled code contains **GC barriers** *≈ small snippets of code the JIT inserts around object reads and writes so the garbage collector can track what the program is doing. Each collector needs different ones*, which are specific to each collector.¹⁷ The CPU rule exists because C2 uses whatever instructions the training machine offers. A secondary summary of the JEP gives the example of code compiled to use AVX-512 instructions, which can't run on an x64 CPU without them.¹⁷ Nothing crashes on a mismatch. You just lose the speed-up for the code part. The PR also adds a switch, `-XX:-AOTCodeCaching`, to turn off the compiled-code part while keeping the rest of the cache.^{5,17}

So how much faster is it? The JEP's numbers are pre-release figures from the feature's authors, so treat them as claims. On its benchmark applications, the AOT cache *without* compiled code cut startup by roughly 50–

70%. With JEP 544's code included, the cut is roughly 65–80%. For `javac` itself, startup improved by about 75%.^{15,18}

Startup time reduction reported in JEP 544 (pre-release, higher is better)

AOT cache, no code (JDK 25/26)



≈50–70%

AOT cache + code (JEP 544)



≈65–80%

Ranges as reported from the JEP text by [daily.dev](#) and [backendbytes.com](#); bar length shows the top of each range. One secondary summary says the measurements were taken on a 2-core Linux/x64 machine. These are the authors' own pre-release benchmarks, not a prediction for your app.

For comparison, here are independent measurements of the *older* layers. Gunnar Morling measured JEP 483 alone cutting Apache Kafka 4.0's startup from 690 ms to 285 ms (59%), and an Apache Flink job's time to first message from 1.875 s to 0.913 s (51%).¹⁹

WHAT IT MEANS FOR YOU

You don't need to change any code. That's an explicit goal: the feature requires no changes to application or library code.^{15,14} What changes is your build and deploy pipeline, and you can start practising on JDK 25 today, because JEPs 483, 514 and 515 are already there.

In the real world: Spring Boot already supports the AOT cache

Spring Boot documents the AOT cache for Java 25 and above. You extract the executable jar, do a training run that exits once the application context is ready, then start with the cache:²⁰

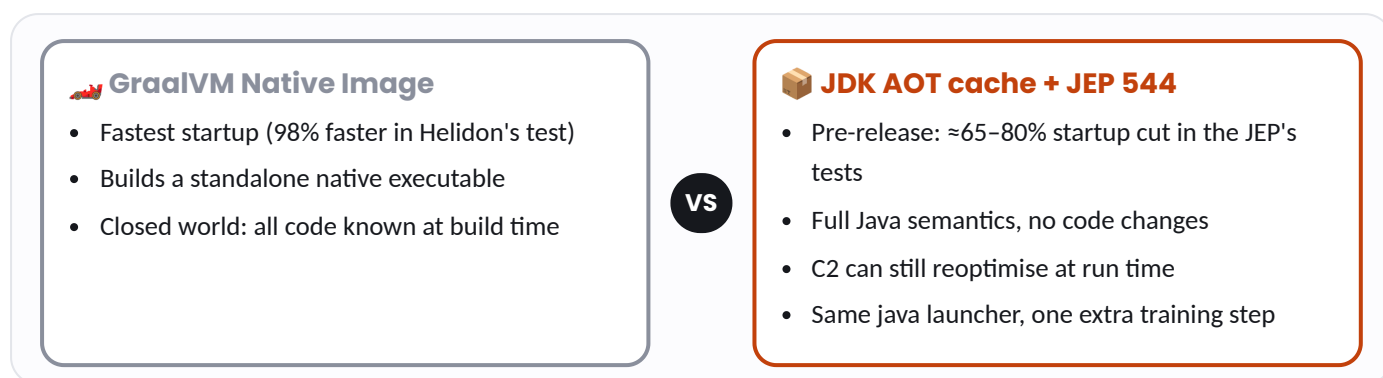
```
java -Djarmode=tools -jar my-app.jar extract --destination application
cd application
java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar my-app.jar
java -XX:AOTCache=app.aot -jar my-app.jar
```

The cache only works with the extracted form of the app and the same Java version that created it.²⁰ Quarkus said in 2025 that it "fully" supports the JDK 24 cache. Helidon measured JEP 483 alone at a 62–67% startup speed-up, against 95–98% for CRaC and 98% for GraalVM Native Image (vendor numbers).¹⁰

Your training run now matters much more. `spring.context.exit=onRefresh` stops the app as soon as the context has started. That's perfect for caching classes, but it never runs your real request paths. Spring Framework's documentation already warns that since Java 25 the cache stores profiles, so it's "recommended to create an AOT cache for an application that experienced a portion of a production-like workflow" rather than using `onRefresh`, which "is designed to optimize only the startup".²¹ With JEP 544 the cache stores compiled code for whatever ran during training, so the same advice matters even more. A training run that only boots the app will give you a fast start but leave the request paths to be warmed up by the JIT as before.

Build the cache where you run. Because AOT code is tied to CPU features and the garbage collector, a cache built on a developer's laptop or on a different instance type may not deliver its code to production. If you ship multi-architecture images (x64 and ARM), you'll need a cache per architecture. If you pick the GC with flags such as `-XX:+UseZGC`, train with the same flags. Classpath rules from JEP 483 still apply: the same JARs, in the same order.^{19,21}

Leyden vs Native Image becomes a real choice. GraalVM Native Image still wins on raw startup. Leyden's pitch is different: keep the normal JVM, normal reflection and a normal C2 compiler at peak, and get most of the startup win.



🌟 Watch out for

- **Not in a build you can download yet.** JEP 544 is *targeted*, but its implementation PR was still open and its CSR still pending when we checked.⁵ JDK 28 itself ships in March 2027.³
- **JDK 28 isn't an LTS.** It gets six months of Oracle support.³ Many teams will first meet JEP 544 in the next LTS, not in JDK 28.
- **A mismatch fails silently.** A different CPU feature set or GC doesn't stop the app. It just drops the AOT code, and your startup gets slower without any error.¹⁵
- **Same JDK, same build.** As with every AOT cache, the cache must be created and used with the same JDK version, and it must be rebuilt whenever your application changes.^{7,20}
- **The numbers are the authors' own.** 65–80% is a pre-release claim on the JEP's benchmarks. Measure your own service before you promise anyone a number.

WHAT HAPPENS NEXT

The implementation PR needs to finish review and land in the JDK 28 codebase. Once it does, you'll be able to try it in a JDK 28 early-access build (build 18 was the latest at the end of September).¹ IDE and tool support will follow; Eclipse's compiler team is already tracking it.² Meanwhile, if you're on JDK 25, setting up the AOT cache today, with a realistic training run, is the best preparation: when you move to a JDK with JEP 544, the same pipeline gets the compiled code for free.

Words from this story

CSR: Compatibility and Specification Review, the OpenJDK approval step for changes to public behaviour such as new options.

Bytecode: the portable instructions in `.class` files that the JVM executes.

Interpreter: executes bytecode one instruction at a time; immediate but slow.

Profile: statistics about how a method behaves at run time, used to guide optimisation.

JIT compiler: compiles hot bytecode to native code while the program runs.

Native code: machine instructions for a specific CPU.

C1: HotSpot's fast, lightly optimising JIT compiler.

C2: HotSpot's profile-guided, heavily optimising JIT compiler.

Inlining: copying a called method's body into its caller to remove the call and enable more optimisation.

Tiered compilation: interpreter, then C1, then C2 as code gets hotter.

Warmup: the slow period after startup before hot code is fully compiled.

Ahead-of-time compilation: compiling to native code before the program runs.

GraalVM Native Image: compiles a whole app into a standalone native executable under a closed-world assumption.

Class Data Sharing (CDS): stores parsed class metadata in a file that later JVMs map in.

AOT cache: the file from a training run holding classes, profiles and now compiled code.

Linked: a class that has been verified and had its references resolved.

Training run: a run on a representative workload whose purpose is to fill the AOT cache.

Deoptimize: discard compiled code whose assumptions failed and fall back to the interpreter.

GC barriers: collector-specific code the JIT inserts around object reads and writes.

STORY 2 Spring's "Patch Thursday": one day a month, every fix at once

SPRING · SECURITY · ANNOUNCED 21 SEP 2026 · FIRST PATCH TRAIN 22 OCT · ~11 MIN READ

For years, a typical month brought the Spring team about six or seven security reports. In April 2026 it brought 482. That isn't a typo, and it isn't a sign that Spring suddenly got worse. It's what happens when AI models start reading open-source code for bugs at industrial scale. Spring's answer, announced on 21 September, is to stop releasing in a rolling two-week wave and ship the entire portfolio on one day a month. The first of those days is Thursday 22 October. If your team runs Spring, it's worth understanding why this happened and how to plan around it.

WHAT HAPPENED

On 21 September, Spring's Michael Minella announced that "the Spring release train has been updated from a two week long train to a single day, the Thursday after the third Monday of the month." The next regular patch train is on **22 October**. September got a milestone-only train on 24 September.²²

The post explains the reason directly: "We have averaged nearly 80 new community provided security reports a month since March, in addition to our own internal scanning with frontier models," and "the rate we are seeing new reports coming in shows no signs of slowing soon." The team says it has fixed over 160 new CVEs and has had to publish as many as 91 security advisories in a single month.²²

Two more changes came with it. All Spring projects now release together, in one drop, so "a 'Patch Thursday' if you will for Spring". And the advisory page at spring.io/security was rebuilt from a rolling blog into a searchable list you can filter by CVE ID, severity and project.^{22,23}

THE BACKGROUND YOU NEED

What a "release train" is

Spring isn't one library. It's a portfolio: Spring Framework at the bottom, then projects such as Spring Data, Spring Security, Spring Integration and Reactor, then Spring Boot, which pulls them together, and Spring Cloud on top. A **release train** *≈ a set of related projects released together on a schedule, so their versions are known to work with each other* is the coordinated release of all of them.

Until now, that train was spread out. In Minella's words, "we spread releases over a two-week period with each level of the Spring portfolio dependency tree shipping on a different day." The lower layers shipped first, because each layer has to build and test against the patched versions of the layers below it.²²

Why staggered releases became a problem

Each of those releases could carry security fixes. A **CVE** *≈ Common Vulnerabilities and Exposures: a public, numbered identifier for a specific security flaw, such as CVE-2022-22965* fixed in Spring Framework on day one was public on day one, but you might not get a Spring Boot version containing it until days later. Minella notes that under the old model, the past two trains "would have had new CVE fixes released every day of each train." For a security

team, that means two weeks of "is today's fix the one we need?" and a window in which a vulnerability is public while the version you actually consume isn't out yet. As the post puts it, "in a world where CVEs are exploited in hours instead of days, it now seems archaic."²²

"Exploited in hours" isn't hyperbole for Spring. In March 2022, a remote code execution flaw in Spring Framework, CVE-2022-22965, nicknamed **Spring4Shell** *≈ a critical 2022 Spring Framework flaw (CVSS 9.8) that let attackers write a web shell to vulnerable servers running on JDK 9+*, was rated **CVSS** *≈ the Common Vulnerability Scoring System: a 0–10 severity score for a vulnerability, where 9.0 and above is "critical"* 9.8. In its update of 31 March 2022, the SANS Internet Storm Center was already reporting exploitation attempts that planted **web shells** *≈ small malicious server-side scripts that give an attacker remote command execution through ordinary HTTP requests*.²⁴

The AI flood

What changed in 2026 is the volume. In June, Minella wrote that "AI models have drastically reduced the level of skill and knowledge required to identify potential code patterns that could be vulnerabilities." In April 2026 the team "received an unprecedented 482 new security reports across 65 scanned projects", against a historical average of about 6.5 a month.²⁵ Sonatype's analysis splits that April total into 370 from Spring's own scanning and 112 from the community, up from 55 in March.²⁶

Not all of it is real. The June post says 37% of the internal scanning results turned out to be duplicates or invalid.²⁵ But enough of it is. On 20 August, Spring published fixes for 91 vulnerabilities in one go. SecurityWeek counted one critical and more than a dozen high-severity flaws among them, and more than 200 Spring vulnerabilities patched in 2026 so far, against 16 in 2025 and 22 in 2024.²⁷

Spring vulnerabilities patched per year



Source: SecurityWeek, 24 Aug 2026. The 2026 figure is 'over 200' as of the 20 August release, so the bar shows 200 as a floor.

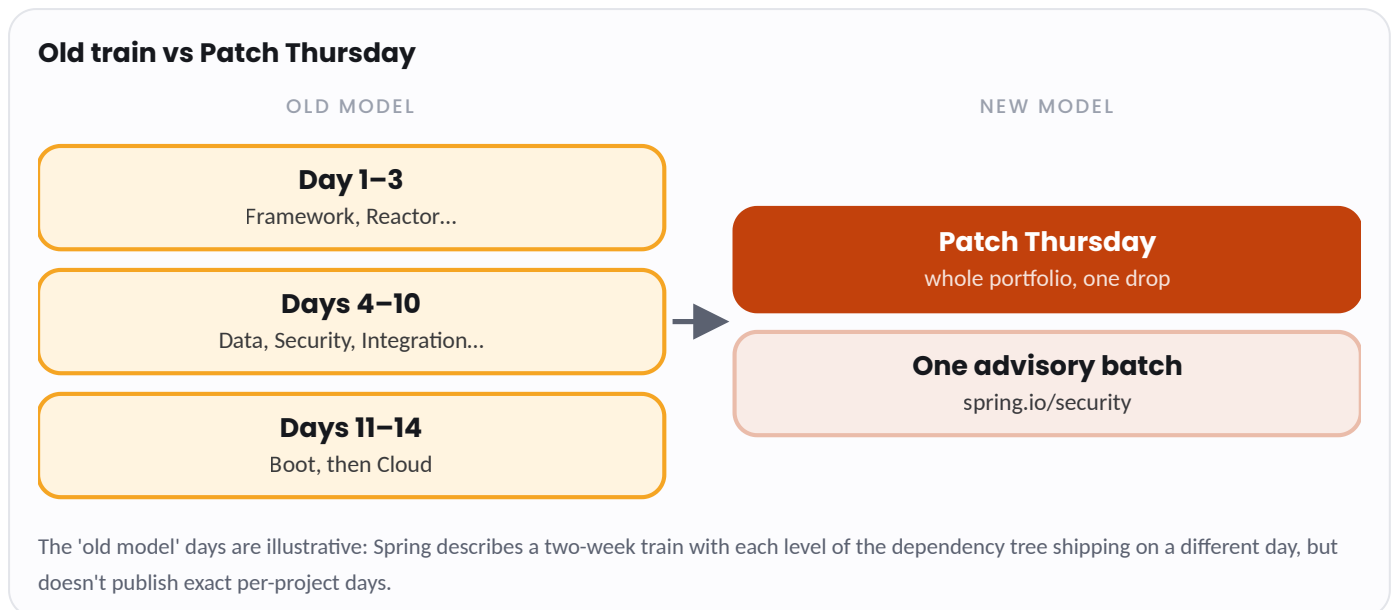
The most serious of the August batch, according to Sonatype, was CVE-2026-59285 in Spring for GraphQL: a critical (CVSS 9.2) unsafe **deserialization** *≈ turning bytes such as JSON back into Java objects. Done unsafely, attacker-controlled input can create objects that run code* flaw that could lead to remote code execution when Jackson 2.x deserialization and paginated GraphQL fields are combined.²⁶ Sonatype counted 209,569 software components affected by the batch.²⁶ Its CTO, Brian Fox, summed up the new problem: "The Spring team has done exactly what we want a maintainer to do here... Now the question is how quickly those fixes make their way through the rest of the software supply chain."²⁶

That's the problem Patch Thursday addresses. The bugs are being fixed. The bottleneck has moved to you, the consumer, and to how predictably you can take the fixes in.

HOW IT WORKS

One day, everything

Under the new model, every Spring project's patch releases reach **Maven Central** *≈ the main public repository from which Maven and Gradle download Java libraries* on the same day. Spring deliberately kept the date most teams already knew: the Thursday after the third Monday of the month, "the same day of the month we have released Spring Boot for years now."²²



Here's what the rule gives for the next few months. We worked these dates out from the rule. Spring has only confirmed 22 October, and the post says nothing about holiday exceptions.²²



Patch, minor, major: what lands on the day

Spring versions follow the familiar `major.minor.patch` pattern. A **patch release** *≈ a version that only bumps the last number (4.1.1 → 4.1.2) and is meant to contain bug and security fixes, not new features* is what Patch Thursday is about. **Minor releases** *≈ versions that bump the middle number (4.1 → 4.2) and bring new features while staying broadly compatible* come on their own schedule. Spring's next ones, the "November feature releases", are what September's

milestones were preparing.²² For example, Spring Boot 4.2.0-M2 and Spring Cloud 2026.0.0-M1 ("Paddington", built on Boot 4.2.0-M2) shipped in the 24–25 September milestone train.²⁸

Patches only flow to versions that are still supported. Spring's policy gives each Spring Boot minor release at least 13 months of free open-source support, or 25 months with a commercial subscription.²⁹ In practice, as of this month, only Spring Boot 4.1.x and 4.0.x get free updates. Boot 3.5's free support ended in June 2026, and 4.0's ends in December 2026.³⁰

The new advisory page

Ninety-one advisories don't fit in a blog feed. The rebuilt spring.io/security lets you filter by project (including Micrometer and Reactor), by **generation** ≈ Spring's term for a supported release line of a project, such as Spring Boot 4.1.x and by severity, and each entry lists the fixed versions.²³ If you only run Spring Boot 4.1 with Security and Data JPA, you can now see just the advisories that apply to you.

WHAT IT MEANS FOR YOU

Put a recurring upgrade slot in the calendar. The biggest benefit of Patch Thursday is predictability. You know when fixes will arrive, so you can schedule the work: for example, an upgrade PR on the Friday and a deployment the following week. If you use Spring Boot's dependency management, bumping the Boot version usually brings in the matching Framework, Security, Data and Reactor patches together. That matters more now that they are all released on the same day.

Automate the boring part. Dependency bots can follow the rhythm. Here's an illustrative Dependabot setup using documented options: a weekly check on Fridays, with all Spring updates grouped into one pull request.³¹

```
# .github/dependabot.yml (illustrative)
version: 2
updates:
  - package-ecosystem: "maven"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "friday"
      time: "06:00"
      timezone: "Asia/Kolkata"
    groups:
      spring:
        patterns: ["org.springframework*", "io.projectreactor*"]
        update-types: ["minor", "patch"]
```

Get onto a supported line. A predictable patch day is worthless if your version no longer receives patches. If you're still on Boot 3.x, you're already outside free support. Dan Vega's advice is to go to 4.1 rather than 4.0, because 4.0's free support ends in December.³⁰ The alternative is paying: VMware Tanzu Spring offers

commercial support, including what Minella calls "Day 0 access to security patches" through its enterprise repository.²⁵

Triage by severity, not by count. "91 CVEs" sounds terrifying, but the June post notes that most of the flood is medium or low severity.²⁵ Use the severity filter on spring.io/security, check which advisories touch the modules you actually use, and keep emergency effort for the criticals. Sonatype's point stands, though: the work of finding and fixing bugs has accelerated, and the work of consuming fixes has to keep up.²⁶

In the real world: the week of 19 October

For many teams, the week of 19 October will be a busy one. Oracle's October Critical Patch Update lands on Tuesday 20 October (the same update that moves Oracle JDK 21 to paid licensing, as we covered in Issue #1). Spring's first Patch Thursday follows two days later, on the 22nd.²² If you were going to touch your runtime and your framework anyway, plan one combined upgrade and test cycle instead of two.

Watch out for

- **The date is when artifacts reach Maven Central**, per Spring's post.²² Mirrors and internal proxies (Artifactory, Nexus) may lag behind.
- **No word on emergencies.** The post doesn't say whether a critical, actively exploited flaw could ship outside the monthly train. Don't assume it can't.
- **Free patches only for supported lines.** Patch Thursday helps Boot 4.0 and 4.1 users. It does nothing for an unsupported 3.x app.³⁰
- **Volume isn't severity.** Most of the new reports are medium or low. Read the advisory before you panic-upgrade.²⁵

WHAT HAPPENS NEXT

The first real test is **22 October**. Then the November feature releases arrive, bringing the next generation of projects, such as Spring Boot 4.2 and Spring Cloud 2026.0, whose milestones shipped in September.^{22,28} At the end of December, Spring Boot 4.0 drops out of free support.³⁰ The bigger open question is the one Sonatype and DevOps.com raise: if AI keeps finding bugs at this rate, will monthly batches be enough, or will teams be pushed toward continuous patching?³² For now, Spring has chosen predictability, and given you a date to plan around.

Words from this story

Release train: related projects released together on a schedule, so their versions work with each other.

CVE: a public, numbered identifier for a specific security flaw.

Spring4Shell: CVE-2022-22965, a critical 2022 Spring Framework remote code execution flaw.

CVSS: a 0–10 severity score for vulnerabilities; 9.0+ is critical.

Web shell: a malicious server-side script giving an attacker remote command execution over HTTP.

Deserialization: turning bytes such as JSON back into objects; unsafe deserialization can let input run code.

Maven Central: the main public repository Maven and Gradle download libraries from.

Patch release: a last-number version bump with fixes only.

Minor release: a middle-number version bump with new features, broadly compatible.

Generation: Spring's term for a supported release line, such as Boot 4.1.x.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **JEP 535, Shenandoah generational mode by default:** targeted to JDK 28, it makes generational mode the default for the Shenandoah garbage collector and deprecates the non-generational mode.³
- **JEP 540, a JSON API in the JDK:** a "Simple JSON API" for parsing and generating JSON without external libraries is headed to JDK 28 as an incubator module.³
- **Jakarta EE 12 schedule:** Ivar Grimstad shared target dates of 1 December for the Core Profile, 31 March for the Web Profile and 15 May for the full Platform.¹
- **Eclipse JNoSQL graduates:** the NoSQL/Jakarta Data project left incubation to become a full EE4J project, and version 1.19.0 added ScyllaDB support.¹
- **JobRunr 9.0.0:** the background-job library added a Gantt-chart history view, Micronaut 5 and Quarkus 3.40 support, and warnings for oversized serialized jobs.¹
- **Lathe:** a new Java language server (LSP) that builds its project model straight from your Maven build, aiming to end IDE import headaches.¹
- **Spring's September milestones:** Spring Boot 4.2.0-M2 adds a property to choose Micrometer or OpenTelemetry observation conventions, and Spring Cloud 2026.0.0-M1 "Paddington" is the first cloud milestone on Boot 4.2.²⁸

1. [Java News Roundup: JobRunr 9, OpenXava 8, Quarkus, LangChain4j, JNoSQL, Introducing Lathe](#), InfoQ, 5 Oct 2026.
2. [\[28\] JEP 544: Ahead-of-Time Code Compilation, issue #5476](#), eclipse-jdt/eclipse.jdt.core on GitHub, opened 5 Oct 2026 (quotes the JEP summary).
3. [Java 28 starts to take shape](#), Paul Krill, InfoWorld, 24 Sep 2026.
4. [Java News Roundup: New OpenJDK JEPs, CDI 5.0, Spring, Open Liberty, RefactorFirst, ADK for Kotlin](#), InfoQ, Sep 2026.
5. [8380476: Implement JEP 544: Ahead-of-Time Code Compilation, PR #30778](#), openjdk/jdk on GitHub (checked 6 Oct 2026). The JEP itself is at [openjdk.org/jeps/544](#); it was not reachable from our research environment, so its details here come from the sources cited.
6. [JEP 515: Ahead-of-Time Method Profiling](#), OpenJDK (delivered in JDK 25).
7. [JEP 483: Ahead-of-Time Class Loading & Linking](#), OpenJDK (delivered in JDK 24).
8. [JEP 410: Remove the Experimental AOT and JIT Compiler](#), OpenJDK (JDK 17).
9. [RFR: 8264805: Remove the experimental Ahead-of-Time Compiler](#), Vladimir Kozlov, build-dev mailing list, 8 Apr 2021.
10. [Java Applications Can Start 40% Faster in Java 24](#), Karsten Silz, InfoQ, 18 Mar 2025.
11. [Project Leyden](#), OpenJDK project page.

12. **JEP 514: Ahead-of-Time Command-Line Ergonomics**, OpenJDK (delivered in JDK 25).
13. **OpenJDK Project JEPs map**, Chris Newland (JEP status listing, checked 6 Oct 2026).
14. **JEP draft 8335368: Ahead-of-Time Code Compilation**, John Rose, OpenJDK (the draft that became JEP 544; version last updated Oct 2025).
15. **JEP 544: Ahead-of-Time Code Compilation**, daily.dev summary of the JEP, Sep 2026.
16. **Java's new AOT cache cuts startup time by up to 80%**, zeli.app (quotes the JEP summary).
17. **JEP 544 lleva la compilación AOT nativa a la JVM de Java**, Ecosistema Startup (Spanish-language technical summary of the JEP).
18. **JVM Cold Start in 2026: Leyden vs GraalVM Native Image vs CRaC**, backendbytes.com, updated 25 Sep 2026.
19. **JEP 483: Ahead-of-Time Class Loading & Linking**, Gunnar Morling, 27 Mar 2025.
20. **AOT Cache**, Spring Boot reference documentation.
21. **JVM AOT Cache**, Spring Framework 7.1 reference documentation.
22. **Releasing Spring for Modern Challenges**, Michael Minella, spring.io, 21 Sep 2026.
23. **Spring Security Advisories**, spring.io (checked 6 Oct 2026).
24. **Spring Vulnerability Update: Exploitation Attempts CVE-2022-22965**, SANS Internet Storm Center, 31 Mar 2022.
25. **Spring and Security In The Times Of AI**, Michael Minella, spring.io, 1 Jun 2026.
26. **91 Spring CVEs: The AI Vulnerability Consumption Problem**, Sonatype, via Security Boulevard, 21 Aug 2026.
27. **91 Vulnerabilities Patched in Spring Application Framework**, Eduard Kovacs, SecurityWeek, 24 Aug 2026.
28. **Spring News Roundup: Second Milestone Releases of Boot, Framework, Data, Security, Integration**, InfoQ, 29 Sep 2026.
29. **Spring Support Policy**, spring.io.
30. **Spring Boot End of Life**, Dan Vega.
31. **Optimizing the creation of pull requests for Dependabot version updates**, GitHub Docs.
32. **Report Shines Spotlight on 91 Vulnerabilities Fixed in Latest Update to Spring Framework**, Mike Vizard, DevOps.com, 25 Aug 2026.